## void *get_register(const char* reg, int size)*

Returns pointer to register

**Error**:

returns *Null* for unknown register or mismatching size

## find_any_value( const char* s)

Returns the value of a *register* or *immediate*.

Supports :

```
find_any_value("eax") → // works fine on extended registers
find_any_value("ax") → // as well as normal registers
find_any_value("ah") → // or byte registers
find_any_value("-32") → // > -32
find_any_value("'a'") → // > 97
find_any_value("[100]") → // > export 4bytes integer starting from mem[100]
```

**function error handling** :

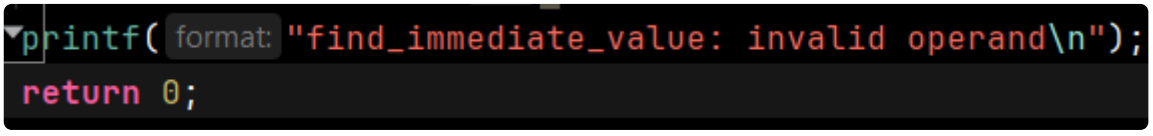
if unsopported operand is passed into the function we'll be alerted in console

## int find_immediate_value(const char* s)

Supports :

```
find_immediate_value("'s'")  // characters
find_immediate_value("-32")  // negative numbers
find_immediate_value("1000") // pos numbers
```

**Errror** :




## *int* find_mem32_value(*int* addr)

```
returns signed 4byte integer starting from mem[addr]
```

*mem16 mem8 and unsigned versions will be added too!*

## push_int (int value)

```
Pushes a 4bytes integer into the stack (managed in `memory[]`).
Stack pointer (`esp`) is also updated accordingly.
```

**Error handling** :
Prints `"push_int: stack overflow"` if the stack is full.

## pop_int ( )

- **Error:**
  Prints `"pop_int: stack underflow"` if the stack is empty.
- **Returns:**
  The popped value, or -1 on error.

*note : we also have push_handler and pop_handler for register values dont mix them up :}*

## lstrip(char* line)`

- Removes leading whitespace from a line *inplace* [used for our custom parsing]
-

## *char* first_char(*char** line)

```
Returns the first non-whitespace character from a string.
usecase : [determining whether a line is started with # or not]
```

## int char_to_i(char* arg)

```
returns ascii code of character
```

Edge cases :
'\0'
'\t'
'\\'

## show_all_registers( )

```
print hexadecimal value of all 4byte registers
```

## show_flags ( )

```
Displays binary values of all flags (zf, sf, cf, of)
```

## clear_flags ()

```
Sets all flags to 0
```

## is_num(char* arg)

edge cases :
is_num("-32")
is_num("+78")

## is_mem_addr(char* arg)

edge cases :
is_mem_addr("[HI]") -> returns 1

## is_immediate

for numbers and characters in form of '\n' 's' '\t' ...

## is_char(char* arg)

edge cases:
is_char ("'\0")
is_char("")
is_char("'\t")